

WSP Programming Language

by

Tatiya Seehatrakul (st124875)

Puskar Adhikari (st125098)

Shreeyukta Pradhanang (st125168)

A Project Report Submitted in
AT70.07 Programming Languages and Compilers

Instructor: Prof. Phan Minh Dung

Teacher Assistant: Akraradet Sinsamersuk

Asian Institute of Technology
School of Engineering and Technology
Thailand
May 2025

ACKNOWLEDGMENTS

We express our sincere gratitude to Professor Phan Minh Dung for his invaluable guidance and expertise throughout the AT70.07 Programming Languages and Compilers course. His insightful direction has been instrumental in my academic growth and the success of this project. The authors are equally grateful to Mr.Akraradet Sinsamersuk, the teaching assistant, for his unwavering support, constructive feedback, and dedication, which greatly enhanced the development process of this work. In addition, we acknowledge the encouragement and collaborative spirit of our peers, whose contributions were essential to the successful completion of the WSP Programming Language project. Their collective efforts have made this endeavor both rewarding and enriching.

ABSTRACT

This project presents the design and implementation of a programming language tailored for educational and scripting purposes. The language features a C-like syntax with strongly typed variables (`int`, `float`, `bool`, `string`), immutable constants, and functions supporting pass-by-value and pass-by-reference parameters. It includes control structures (`if`, `while`), arithmetic and comparison operators, and a unary minus operator, with a `print` statement. Implemented by leveraging the SLY (Sly Lex-Yacc) library for parsing and processing the expressions, ensuring efficient and accurate computation, the project addresses challenges such as parameter binding and operator precedence to ensure correct semantics. A formal grammar defines the language's syntax, providing a clear specification for future extensions. The result is a simple yet expressive language suitable for learning programming concepts and building small-scale applications.

CONTENTS

	Page
ACKNOWLEDGEMENTS	2
ABSTRACT	3
CHAPTER 1 INTRODUCTION	5
CHAPTER 2 IMPLEMENTATION AND RESULTS	9
2.1 Implementation	9
2.1.1 Lexical Analysis (Tokenizer)	9
2.1.2 Grammar and Parsing	9
2.1.3 Memory and Symbol Table	10
2.1.4 AST Structure	10
2.1.5 User Interface Integration	10
2.1.6 GUI Snapshots	12
2.1.7 Function Support and Parameter Passing	15
2.2 Git Repository	15
CHAPTER 3 CONCLUSION	16
3.1 Future Work	17
3.2 Team Contributions	17

CHAPTER 1

INTRODUCTION

This project presents the design and implementation of a simplified programming language built using Python, aimed at educational and scripting purposes. The language is designed to be clear, functional, and easy to understand, incorporating essential features commonly found in modern programming languages such as Python and Java.

The language supports four fundamental data types: `int`, `float`, `boolean`, and `string`, with static typing to simplify implementation and enhance flexibility. It enables arithmetic operations using the basic operators (`+`, `-`, `*`, `/`) while respecting standard precedence rules. In addition, the language provides support for boolean expressions (equality and inequality comparisons) and string concatenation.

Core programming constructs include variable assignments, conditional branching through `if-else` statements, iterative control using `while` loops, function abstraction with parameter passing (non-recursive), and a built-in `print()` function for output.

The implementation is developed entirely in **Python**, utilizing the **PLY (Python Lex-Yacc)** library to construct the compiler components. The system is organized into three main modules:

1. A **lexer** to tokenize source code using defined lexical rules.
2. A **parser** to construct a syntax tree based on grammar rules.
3. An **interpreter** to execute the parsed syntax tree and produce output.

This project was developed by a team of three members. The outcome includes both the language implementation and a comprehensive report detailing the grammar, syntax, and semantics of the language. The result is a compact yet expressive system suitable for learning programming concepts and exploring language design.

Grammar of WSP Language

The following grammar defines the syntax for the WSP programming language. The grammar is presented in Backus-Naur Form (BNF) notation, with non-terminals in angle brackets and terminals in boldface.

Statements

```
⟨statements⟩ ::= ⟨statement⟩;⟨statements⟩
                | ⟨statement⟩;
                | ⟨statement⟩
                | ⟨type⟩IDENTIFIER=⟨expr⟩;
                | IDENTIFIER=⟨expr⟩;
                | ⟨type⟩IDENTIFIER;
                | print(⟨expr⟩);
                | if(⟨expr⟩){⟨statements⟩}
                | if(⟨expr⟩){⟨statements⟩}else{⟨statements⟩}
                | while(⟨expr⟩){⟨statements⟩}
                | void func IDENTIFIER(⟨param_list⟩){⟨statements⟩}
                | ⟨type⟩ func IDENTIFIER(⟨param_list⟩){⟨statements⟩}
                | return⟨expr⟩;
                | IDENTIFIER(⟨arg_list⟩);
```

Parameters and Arguments

$\langle \text{param_list} \rangle ::= \text{const} \langle \text{type} \rangle \text{IDENTIFIER}$
| $\langle \text{type} \rangle \text{IDENTIFIER}$
| $\langle \text{param_list} \rangle, \text{const} \langle \text{type} \rangle \text{IDENTIFIER}$
| $\langle \text{param_list} \rangle, \langle \text{type} \rangle \text{IDENTIFIER}$
| ϵ
 $\langle \text{arg_list} \rangle ::= \langle \text{expr} \rangle$
| $\langle \text{arg_list} \rangle, \langle \text{expr} \rangle$
| ϵ

Types

$\langle \text{type} \rangle ::= \text{int}$
| **float**
| **bool**
| **string**

Expressions

$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle + \langle \text{expr} \rangle$
| $\langle \text{expr} \rangle - \langle \text{expr} \rangle$
| $- \langle \text{expr} \rangle$
| $\langle \text{expr} \rangle * \langle \text{expr} \rangle$
| $\langle \text{expr} \rangle / \langle \text{expr} \rangle$
| $\langle \text{expr} \rangle < \langle \text{expr} \rangle$
| $\langle \text{expr} \rangle <= \langle \text{expr} \rangle$
| $\langle \text{expr} \rangle > \langle \text{expr} \rangle$
| $\langle \text{expr} \rangle >= \langle \text{expr} \rangle$
| $\langle \text{expr} \rangle == \langle \text{expr} \rangle$
| $\langle \text{expr} \rangle != \langle \text{expr} \rangle$
| **IDENTIFIER**($\langle \text{arg_list} \rangle$)
| **INT_LITERAL**
| **FLOAT_LITERAL**
| **BOOL_LITERAL**
| **STRING_LITERAL**
| **IDENTIFIER**
| $(\langle \text{expr} \rangle)$

Precedence and Associativity

The following operator precedence and associativity rules apply:

- **==, !=**: Left associative
- **<, <=, >, >=**: Left associative
- **+, -**: Left associative
- ***, /**: Left associative
- **Unary -**: Right associative

CHAPTER 2

IMPLEMENTATION AND RESULTS

2.1 Implementation

The PLC-Language compiler was implemented using the **SLY** (Sly Lex Yacc) library in Python, which simplifies the construction of lexical analyzers and parsers. The overall implementation followed a modular architecture, with core compiler phases mapped to separate components.

2.1.1 Lexical Analysis (Tokenizer)

Lexical analysis was handled in the `lexica.py` file using SLY's `Lexer` class. The lexer defines tokens for identifiers, numbers, operators (+, -, *, /), logical expressions (==, !=, <, <=, etc.), and language keywords such as `if`, `else`, `const`, `int`, `float`, `print`, and `return`. It also captures and ignores whitespace and comments. The lexer outputs tokens for use in the parsing stage.

2.1.2 Grammar and Parsing

The grammar of the language is implemented in the `ASTParser` class inside `parsers.py`. This parser defines rules for:

- Variable declarations (with or without initial values)
- Assignments
- Print statements (`print(x);`)
- Conditional execution (`if (x > 0) { ... } else { ... }`)

Operator precedence is handled through the precedence tuple, which defines associativity and evaluation order, including support for unary minus (`-x`).

All parsed statements are translated into an **Abstract Syntax Tree (AST)** structure that enables semantic analysis and further interpretation or compilation.

2.1.3 Memory and Symbol Table

A custom Memory class was developed in `memory.py` to simulate variable storage. This includes:

- Scope management
- Definitional Parameter passing
- Type-checking logic
- Error handling for re-declarations or invalid assignments

2.1.4 AST Structure

AST nodes were constructed dynamically from parser actions. Each statement results in a tuple like:

```
(‘declare’, ‘int’, ‘a’, (‘number’, 5), line_number, is_const=True)
(‘assign’, ‘a’, (‘binary_op’, ‘+’, (‘id’, ‘a’), (‘number’, 2)), line_number)
```

This structure simplifies execution, debugging, and code generation.

2.1.5 User Interface Integration

To enhance usability and facilitate interaction with the compiler, a simple graphical user interface (GUI) was developed using **Qt Designer**. This interface provides an accessible way for users to write, test, and evaluate their code without relying solely on the command-line interface. The GUI consists of two primary text areas with two interactive buttons to facilitate basic control.

- A **code editor box** (`code_input`, `QTextEdit`) where users can enter or paste their source code. It supports multi-line input and is styled for readability.
- An **output console** (`output_console`, `QTextEdit`) where compilation results or error messages from the compiler are displayed.
- A **Run button** (`run_button`, `QPushButton`) to execute the code entered in the editor and display the corresponding output or errors in the console.
- A **Clear button** (`clear_button`, `QPushButton`) to clear the input and output console, allowing users to reset the display for a new session.

To support quick demonstrations and learning, the GUI includes a set of example buttons under the section **Try Examples**, each linked to a specific pre-defined code snippet that is automatically inserted into the code editor:

1. **Basics** (basic_ex_button): Demonstrates variable declarations, arithmetic, and string concatenation.

```
int a = 10;
float b = 10.5;
string c = "hello";
print(a + 10);
print(b + 1.1);
print(c + " world");
```

2. **If-Else** (if_else_button): Shows conditional branching based on a numerical comparison.

```
int num = 5;
if (num > 0) {
    print("Number is positive");
} else {
    print("Number is negative or zero");
}
```

3. **While** (while_button): Demonstrates loop control using a while loop.

```
int i = 1;
while (i <= 5) {
    print(i);
    i = i + 1;
}
```

4. **Function** (function_button): Presents a user-defined function with typed parameters.

```
int func swap(int x, int y) {
    int temp = x;
    x = y;
    y = temp;
}
int p = 10;
int q = 20;
swap(p, q);
print(p);
print(q);
```

Each button calls a centralized method, `add_examples(button: int)`, which inserts the corresponding code snippet into the editor, supporting modular control, ease of maintenance, and quick access to ready-to-run examples. The interface is minimal and intuitive, serving as a user-friendly frontend to the compiler and making the system accessible to learners without command-line experience.

2.1.6 GUI Snapshots

The following screenshots highlight the main features of the graphical interface, including code input, output display, and example execution. They demonstrate how users interact with the system in a visual and accessible way.

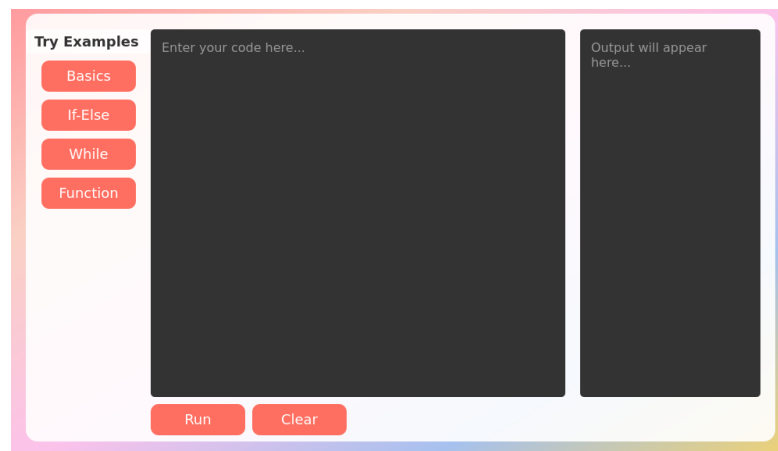


Figure 2.1 *Empty UI layout with editor and output panels.*



Figure 2.2 *Printing variables of int, bool, float, and string types.*

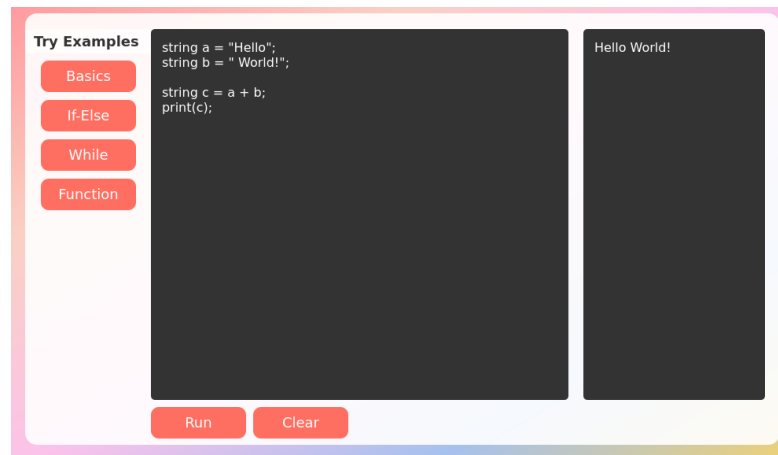


Figure 2.3 String concatenation using the + operator.

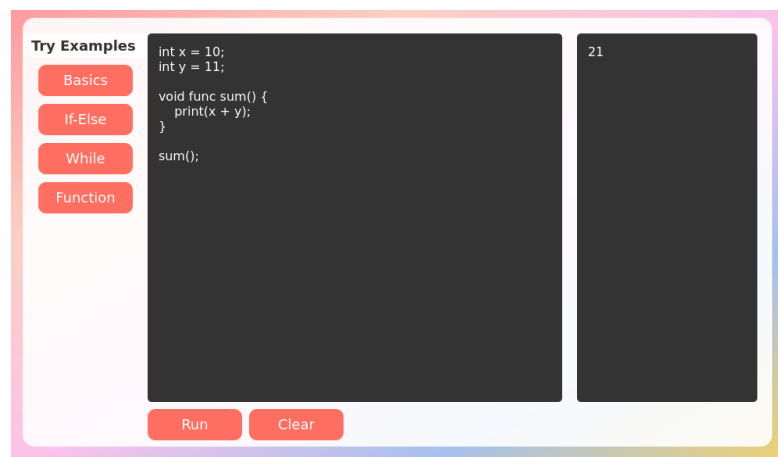


Figure 2.4 Simple function printing sum of two globals.



Figure 2.5 Basic while-loop printing numbers 0 to 4.

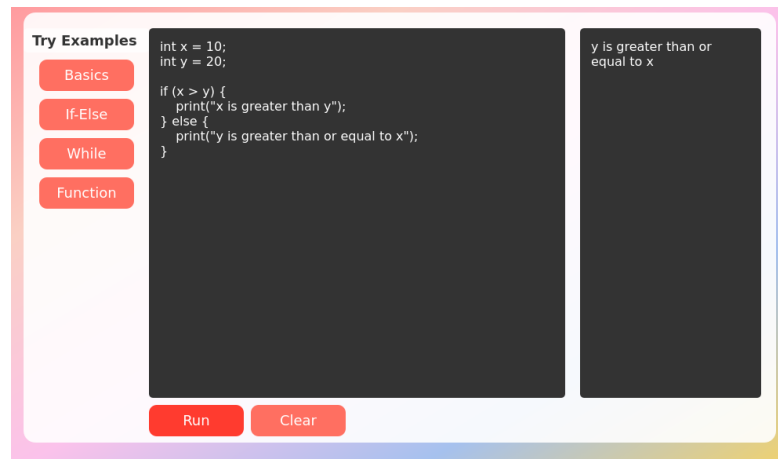


Figure 2.6 Conditional if-else check between two integers.

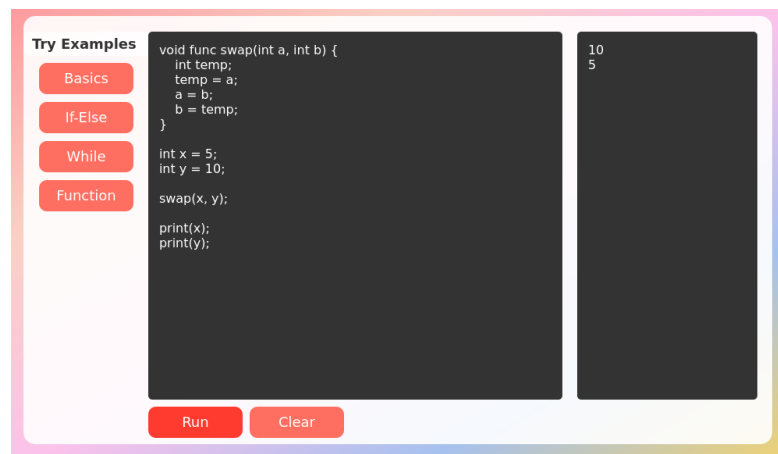


Figure 2.7 Swap function showing parameter passing by value.

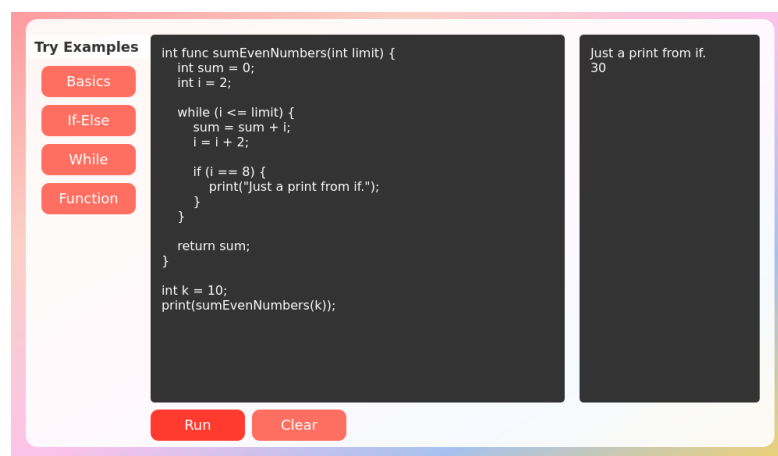


Figure 2.8 Function with loop and nested if-statement.

2.1.7 Function Support and Parameter Passing

The PLC-Language compiler supports both typed functions and parameter passing, distinguishing between mutable ('var') and immutable ('const') parameters. Parameters declared as mutable must be passed using variable identifiers, while constants may accept literals directly.

```
int func add(int a, const int b) {  
    int result = a + b;  
    return result;  
}  
  
int x;  
int temp = 4;  
x = add(temp, 6);  
print(x);
```

The example above demonstrates the following:

- The function add takes two parameters: a (mutable) and b (constant).
- A variable temp is used to satisfy the requirement of passing a mutable variable for a.
- The literal 6 is passed to b as it is marked const.

Expected Output:

```
10
```

This test confirms correct handling of typed parameters, value-returning functions, and expression evaluation within function bodies.

2.2 Git Repository

The complete source code for this project is publicly available on GitHub and can be accessed at: <https://github.com/puskr99/plc-language>. The repository contains all core components of the system, including the full source code and accompanying documentation.

CHAPTER 3

CONCLUSION

This project successfully demonstrates the design and implementation of a simple programming language compiler using Python and the SLY (Sly Lex Yacc) library. Through this work, we explored the core phases of compilation, including lexical analysis, syntax parsing, semantic analysis, and code execution via an abstract syntax tree (AST). Each component was carefully developed to reflect real-world compiler architecture principles in a simplified, educational context.

The language supports essential programming constructs such as variable and constant declarations, assignments, arithmetic expressions, conditional control flow using `if-else`, and function definitions with typed parameters. Special attention was given to the enforcement of rules like immutability of `const` variables and distinction between pass-by-value and pass-by-variable semantics.

An additional graphical user interface (GUI) was integrated using Qt Designer to improve accessibility and allow for more interactive testing. This interface, while not critical to the compiler's operation, provided a user-friendly way to write, run, and debug code within a visual environment.

Key Takeaways: Applying compiler theory in practice significantly deepened our understanding of fundamental concepts such as parsing, tokenization, abstract syntax tree (AST) construction, and error handling. The modular architecture of the compiler facilitated an iterative development process, making it easier to debug and extend functionality over time. Additionally, enforcing strict type rules and mutability constraints reinforced essential principles of sound programming language design.

Overall, the PLC-Language project has provided both theoretical insight and hands-on experience in compiler construction, showcasing how simple language elements can be parsed, interpreted, and executed within a robust framework.

3.1 Future Work

While the current implementation of the PLC-Language compiler demonstrates the essential components of a functional language system, several enhancements can be pursued to improve its capabilities and align it more closely with real-world language features. These include:

- **Loop Constructs:** Incorporating iterative control structures such as for loops and enhanced while loops with support for break and continue statements would significantly increase the expressiveness of the language and allow for more complex algorithmic logic.
- **Nested Scopes and Block-Level Variable Lifetimes:** Introducing support for lexical scoping and variable lifetimes confined to specific code blocks would enable better memory management and allow variables to be reused safely in different scopes, thereby improving the language's structural rigor.
- **Additional Built-in Data Types:** Extending the type system to include native support for strings, arrays, and possibly user-defined data structures would enhance the versatility of the language and broaden its applicability in diverse programming scenarios.
- **Intermediate Code or Bytecode Generation:** Developing an intermediate representation or bytecode output would facilitate further optimization opportunities, enable platform-independent execution, and serve as a foundation for implementing a virtual machine or target-specific code generation in the future.

3.2 Team Contributions

- **Tatiya Seehatrakul (st124875)** – Implemented the print function, arithmetic expressions, conditionals, and designed grammar rules using SLY.
- **Puskar Adhikari (st125098)** – Developed type checking, function execution, memory handling, and evaluated expressions and logic.
- **Shreeyukta Pradhanang (st125168)** – Built the graphical user interface, handled string concatenation, created test programs, and documentation.